Software Requirements Specification (SRS)

Animal Rescue Management System

Course: CSE336 — Software Design Patterns
Version: 1.1

1. Introduction

1.1 Purpose

This document defines the functional and non-functional requirements of the Animal Rescue Management System. It is a reference for implementation, testing, evaluation, and future development.

1.2 Project Definition

The system is a Java desktop application (Swing GUI) that gives an animal rescue organization one place to manage rescued animals, adopters, donations, volunteers, rescue events, communication, and system users.

1.3 Scope

In scope: Animal, Adopter, Donation, Volunteer, Rescue Event, Communication, and User management, plus a main control panel. Data is stored **in memory only**; there is no database and no persistence across restarts.

1.4 Definitions

Term	Definition
CRUD	Create, Read, Update, Delete
MVC	Model-View-Controller architectural pattern
GoF	"Gang of Four" design patterns
Repository	Component that manages access to stored records
Strategy / Observer / Decorator / Factory / Singleton	GoF design patterns used in the project

2. Overall Description

2.1 Product Perspective

A standalone desktop application. Modules are reached through a main control panel and follow an MVC-style structure (`com.rescue.model` , `view` , `controller`) supported by repositories, services, and a factory.

2.2 Users

Role	Responsibility
Rescue Staff Member	Animal records
Adoption Coordinator	Adopter records
Donation Manager	Donation records
Volunteer Coordinator	Volunteer records, skills, availability, matching
Event Coordinator	Events and volunteer registrations
System Administrator	User records

These are intended roles; the current implementation does not enforce separate permission levels per role.

2.3 Operating Environment

Windows, Linux, or macOS; JDK 17 or newer; Java Swing; Apache Maven; any Java IDE.

2.4 Assumptions and Dependencies

- Java is installed and the app is started from its main entry point (`Main`).
- Data lives only for the running session.
- No external services: message delivery uses a console gateway, not real email/SMS providers.

3. Functional Requirements

3.1 Animal Management

- **FR-01** Add an animal (name, species, age in years and months, health status, adoption availability, description).
- **FR-02** Display all animals and view an animal's details.
- **FR-03** Edit animal information.
- **FR-04** Delete an animal.
- **FR-05** Change adoption availability and health status.

3.2 Adopter Management

- **FR-06** Register an adopter (name, email, phone).
- **FR-07** View adopters.
- **FR-08** Update adopter information.
- **FR-09** Delete an adopter.

3.3 Donation Management

- **FR-10** Record a donation for a donor with an amount and receipt text.
- **FR-11** Support online, cheque, and in-kind donation types.

- **FR-12** Support optional recurring and "in memory of" decorations.
- **FR-13** View recorded donations.

3.4 Volunteer Management

- **FR-14** Register a volunteer with contact details, skills, availability, role, and certificates. Skills and availability are chosen from the options in `VolunteerOptions` (`SKILLS`, `AVAILABILITIES`).
- **FR-15** View, update, and delete volunteers.
- **FR-16** Match volunteers by skill or by availability using `VolunteerMatchingStrategy` (`SkillMatchingStrategy`, `AvailabilityMatchingStrategy`).
- **FR-17** Track completed volunteer hours.
- **FR-18** Prevent deleting a volunteer who has event registrations.

3.5 Rescue Event Management

- **FR-19** Create and update rescue events (name, type, date, location, required skills, hours, description).
- **FR-20** Register a volunteer for an event, recording event, volunteer, status, and registration time.
- **FR-21** Prevent duplicate registration of the same volunteer for the same event.
- **FR-22** View events and their registrations.
- **FR-23** Prevent deleting an event that has registrations.

3.6 Communication Management

- **FR-24** Send messages through email, SMS, or social post strategies.
- **FR-25** Support recipient types (including public audience and donors) with default channels per type.
- **FR-26** Keep a history of sent messages and an activity feed.
- **FR-27** Provide observer components for animal-status changes, donations, and event updates. *Note:* the `connect*` hooks exist, but controllers do not currently publish these events in the normal startup path; end-to-end automatic notification must not be claimed until wired and tested.

3.7 User Management

- **FR-28** Add, view, update, and delete user records (username, password, role). Login/authentication enforcement is not specified here beyond what is implemented.

3.8 Control Panel

- **FR-29** Provide a main control panel that opens each management module.

4. Non-Functional Requirements

- **Usability:** screens are reachable from the control panel; input fields are clearly labeled; combo boxes constrain species, health status, skills, and availability.
- **Reliability:** invalid input is reported without crashing the application; duplicate registrations and deletions that would orphan registrations are rejected; records stay consistent within a session.
- **Performance:** operations complete in reasonable time for in-memory data of project size.
- **Maintainability:** models, views, controllers, repositories, and services are separated; patterns reduce coupling.
- **Portability:** runs on any OS with JDK 17+.

5. System Design

5.1 MVC Architecture

- **Model** (`com.rescue.model`): entities such as `Animal` , `Adopter` , `Donor` , `Volunteer` , `RescueEvent` , `EventRegistration` , `Donation` , `User` .
- **View** (`com.rescue.view`): Swing screens, one per module, plus `RescueTheme` .
- **Controller** (`com.rescue.controller`): handles user actions and coordinates views, repositories, and services.

5.2 Main Components

Component	Package	Role
Repositories	`com.rescue.repo`	`Repository<T>` interface, generic `LookupTable<T>` , per-entity repositories over `DataStore`
Factory	`com.rescue.factory`	`DonationFactory`

Component	Package	Role
Services	`com.rescue.service`	`CommunicationModule`, `Subject` / `Observer`
Strategies	`com.rescue.service.strategy`	Channel and volunteer-matching strategies, `Notifier`
Observers	`com.rescue.service.observer`	Event records and observer classes

5.3 Key Relationships

- `VolunteerController` uses `VolunteerRepository`, `RescueEventRepository`, `EventRegistrationRepository`, and `VolunteerMatchingStrategy`.
- `EventController` uses `RescueEventRepository`, `VolunteerRepository`, and `EventRegistrationRepository`.
- `EventRegistration` references a `RescueEvent` (`eventId`) and a `Volunteer` (`volunteerId`).

6. Design Patterns

Phase 1 uses **MVC plus five GoF patterns**. MVC is an architectural pattern, not a GoF pattern.

#	Element	Type	Where used
1	MVC	Architectural	Models, Swing views, controllers
2	Singleton	GoF	`DataStore` (eager), `AppConfig` (lazy)
3	Factory	GoF	`DonationFactory` creates online, cheque, in-kind donations
4	Decorator	GoF	`DonationDecorator`, `RecurringDonation`, `InMemoryOfDonation`
5	Strategy	GoF	`ChannelStrategy` (email, SMS, social post); `VolunteerMatchingStrategy` (skill, availability)
6	Observer	GoF	`Subject<E>` / `Observer<E>`; animal-status, event-update, thank-you, coordinator, dashboard observers (integration partial, see FR-27)

Phase 2 patterns (Command, Adapter, Facade, Template Method, Iterator, State, Proxy) are planned and not implemented.

7. Data Requirements

Entity	Attributes
Animal	id, name, species, ageYears, ageMonths, healthStatus, availableForAdoption, description
Adopter	id, name, email, phone
Donor	id, name, email, phone
Donation	id, donor, amount, receiptText; type (online/cheque/in-kind); optional recurring / in-memory-of (personName)
Volunteer	id, name, email, phone, skills, availability, role, hours, certificates
RescueEvent	id, name, type, date, location, requiredSkills, hours, description
EventRegistration	id, eventId, volunteerId, status, registeredAt
User	id, username, password, role
SentMessage	recorded communication message
VolunteerOptions	SKILLS and AVAILABILITIES option lists

8. External Interfaces

- **User interface:** Java Swing screens opened from the control panel.
- **Software:** Java (JDK 17+), Swing, Apache Maven.
- **Hardware:** any computer able to run Java; nothing specialized.
- **Other:** no database or external messaging service.

9. Limitations and Future Enhancements

Limitations

- In-memory storage only (`DataStore`); all data is lost when the app closes.

- Messages are delivered to a console gateway, not real providers.
- Observer notifications are not fully wired from controllers.
- User roles are descriptive, not enforced permission levels.

Possible Enhancements

- Database persistence.
- Search and filtering.
- Fuller adoption workflow.
- Reports and statistics.
- Real email/SMS integrations.
- Authentication and role-based permissions.
- Phase 2 design patterns.

10. Conclusion

The Animal Rescue Management System organizes the core work of a rescue organization in a Swing desktop application built on MVC and five GoF patterns (Singleton, Factory, Decorator, Strategy, Observer). It is documented here as implemented: in-memory, with volunteer skills, matching, and event registration rules, and with known limits stated openly.